SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

*I **LOKNATH V (192421439L**) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

		and insights			
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWERS

1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Normalization of Order Relation to 3NF

Given Relation:

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

The relation stores customer, order, and product information together. This can cause data redundancy and insertion, deletion, and update anomalies.

1. Identify Functional Dependencies

Based on the meaning of the attributes, the functional dependencies are:

CustID → CustName

A customer ID uniquely identifies the customer name.

ProdID → ProdName, Price

A product ID uniquely identifies the product name and its price.

(OrderID, ProdID) → CustID, Qty

An order can contain multiple products, so OrderID alone is not sufficient to identify an individual order item. The combination of OrderID and ProdID identifies the quantity ordered and the customer associated with the order.

Therefore, the main functional dependencies are:

CustID → CustName

ProdID → ProdName, Price

(OrderID, ProdID) → CustID, Qty

2. Identify the Candidate Key

Consider:

(OrderID, ProdID)+

From (OrderID, ProdID) → CustID, Qty, we get CustID and Qty.

From CustID → CustName, we get CustName.

From ProdID → ProdName, Price, we get ProdName and Price.

Therefore:

(OrderID, ProdID)+ = {OrderID, ProdID, CustID, CustName, ProdName, Qty, Price}

Thus, **(OrderID, ProdID)** is the candidate key.

3. Check for 2NF

The candidate key is composite:

(OrderID, ProdID)

However:

ProdID → ProdName, Price

ProdName and Price depend only on **ProdID**, which is only part of the candidate key.

Therefore, this is a **partial dependency**, so the relation is not in 2NF.

4. Decompose to Remove Partial Dependencies

Create a separate PRODUCT relation:

PRODUCT(ProdID, ProdName, Price)

Primary Key: **ProdID**

The remaining order information becomes:

ORDER_ITEM(OrderID, ProdID, CustID, Qty)

Primary Key: **(OrderID, ProdID)**

5. Check for Transitive Dependency

In ORDER_ITEM:

OrderID, ProdID → CustID

and:

CustID → CustName

Therefore:

(OrderID, ProdID) → CustID → CustName

This is a **transitive dependency**.

To remove it, create a separate CUSTOMER relation.

6. Final 3NF Decomposition

CUSTOMER(CustID, CustName)

Primary Key: **CustID**

PRODUCT(ProdID, ProdName, Price)

Primary Key: **ProdID**

ORDER_ITEM(OrderID, ProdID, CustID, Qty)

Primary Key: **(OrderID, ProdID)**

Foreign Keys:

CustID references CUSTOMER(CustID)

ProdID references PRODUCT(ProdID)

7. Verification of 3NF

CUSTOMER:

CustID → CustName

Since CustID is the primary key, it satisfies 3NF.

PRODUCT:

ProdID → ProdName, Price

Since ProdID is the primary key, it satisfies 3NF.

ORDER_ITEM:

(OrderID, ProdID) → CustID, Qty

Since (OrderID, ProdID) is the primary key, it satisfies 3NF.

Final Answer:

Original relation:

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Functional dependencies:

CustID → CustName

ProdID → ProdName, Price

(OrderID, ProdID) → CustID, Qty

Candidate Key:

(OrderID, ProdID)

3NF decomposition:

CUSTOMER(CustID, CustName)

PRODUCT(ProdID, ProdName, Price)

ORDER_ITEM(OrderID, ProdID, CustID, Qty)

Thus, the original relation is decomposed into **CUSTOMER, PRODUCT, and ORDER_ITEM**, eliminating partial and transitive dependencies and producing a **3NF design**.

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Normalization of Hospital Patient Relation to BCNF

Given Relation:

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

The relation stores patient, doctor, and ward information in a single table. This can result in redundancy and insertion, deletion, and update anomalies.

1. Identify Functional Dependencies

The functional dependencies are:

PID → PName, DocID, WardNo

A Patient ID uniquely identifies the patient's name, assigned doctor, and ward.

DocID → DocName, Specialization

A Doctor ID uniquely identifies the doctor's name and specialization.

WardNo → WardName

A Ward Number uniquely identifies the ward name.

Therefore, the main functional dependencies are:

PID → PName, DocID, WardNo

DocID → DocName, Specialization

WardNo → WardName

2. Identify the Candidate Key

Consider the closure of PID:

PID+ = {PID}

Using PID → PName, DocID, WardNo:

PID+ = {PID, PName, DocID, WardNo}

Using DocID → DocName, Specialization:

PID+ = {PID, PName, DocID, DocName, Specialization, WardNo}

Using WardNo → WardName:

PID+ = {PID, PName, DocID, DocName, Specialization, WardNo, WardName}

Therefore, PID determines all attributes.

Candidate Key = PID

3. Identify Anomalies

Insertion Anomaly:

A new doctor cannot be stored easily unless a patient is assigned to that doctor. Similarly, a new ward cannot be recorded unless a patient is admitted to it.

Deletion Anomaly:

If the only patient assigned to a particular doctor is deleted, information about that doctor may also be lost. Similarly, deleting the only patient in a ward may remove information about that ward.

Update Anomaly:

If a doctor's specialization changes, the information may have to be updated in several patient records. If the ward name changes, multiple records may also need to be updated.

4. Check for BCNF

A relation is in BCNF if, for every non-trivial functional dependency X → Y, **X must be a superkey**.

Consider:

PID → PName, DocID, WardNo

PID is the candidate key, so this dependency satisfies BCNF.

Now consider:

DocID → DocName, Specialization

DocID is not a candidate key of the original relation because DocID cannot determine PID or WardNo.

Therefore, this dependency violates BCNF.

Also:

WardNo → WardName

WardNo is not a candidate key of the original relation because it cannot determine PID or DocID.

Therefore, this dependency also violates BCNF. Hence,

the original PATIENT relation is **not in BCNF**.

5. BCNF Decomposition

To eliminate the violation caused by:

DocID → DocName, Specialization

create:

DOCTOR(DocID, DocName, Specialization)

Primary Key: **DocID**

The remaining relation is:

PATIENT(PID, PName, DocID, WardNo, WardName)

Next, consider:

WardNo → WardName

This violates BCNF in the remaining relation because WardNo is not a superkey.

Therefore, decompose it into:

WARD(WardNo, WardName)

Primary Key: **WardNo**

The remaining patient relation becomes:

PATIENT(PID, PName, DocID, WardNo)

Primary Key: **PID**

6. Final BCNF Relations

PATIENT(PID, PName, DocID, WardNo)

Primary Key: PID

Foreign Keys: DocID, WardNo

DOCTOR(DocID, DocName, Specialization)

Primary Key: DocID

WARD(WardNo, WardName)

Primary Key: WardNo

7. BCNF Verification

PATIENT:

PID → PName, DocID, WardNo

PID is the candidate key, so PATIENT is in BCNF.

DOCTOR:

DocID → DocName, Specialization

DocID is the candidate key, so DOCTOR is in BCNF.

WARD:

WardNo → WardName

WardNo is the candidate key, so WARD is in BCNF.

Conclusion:

The original relation contains redundancy and suffers from **insertion, deletion, and update anomalies**. The dependencies **DocID → DocName, Specialization** and **WardNo → WardName** violate BCNF because their determinants are not superkeys of the original relation.

The final BCNF decomposition is:

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

This decomposition removes the BCNF violations and reduces data redundancy.

3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

2NF Decomposition of UNIVERSITY ENROLLMENT Relation

Given Relation:

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

The relation stores student and course details along with the grade received by the student.

1. Identify the Functional Dependencies

The functional dependencies are:

SID → SName

A Student ID uniquely identifies the student's name.

CourseID → CourseName, Faculty

A Course ID uniquely identifies the course name and the faculty handling the course.

(SID, CourseID) → Grade

The grade depends on both the student and the course.

Therefore, the main functional dependencies are:

SID → SName

CourseID → CourseName, Faculty

(SID, CourseID) → Grade

2. Identify the Candidate Key

A student can enroll in multiple courses, and a course can have multiple students.

Therefore, **SID alone** cannot uniquely identify a record.

Similarly, **CourseID alone** cannot uniquely identify a record.

The combination of SID and CourseID uniquely identifies an enrollment record.

Therefore:

Candidate Key = (SID, CourseID)

3. Check for Partial Dependencies

The candidate key is composite:

(SID, CourseID)

Now check the non-key attributes.

SID → SName

SName depends only on SID, which is only a part of the composite key.

Therefore, this is a **partial dependency**.

Similarly:

CourseID → CourseName, Faculty

CourseName and Faculty depend only on CourseID, which is also only a part of the composite key.

Therefore, these are also **partial dependencies**.

Hence, the original relation is **not in 2NF**.

4. Decompose the Relation into 2NF

To eliminate the partial dependencies, separate student information, course information, and enrollment information.

STUDENT(SID, SName)

Primary Key: **SID**

Functional Dependency:

SID → SName

COURSE(CourseID, CourseName, Faculty)

Primary Key: **CourseID**

Functional Dependency:

CourseID → CourseName, Faculty

ENROLLMENT(SID, CourseID, Grade)

Primary Key: **(SID, CourseID)**

Functional Dependency:

(SID, CourseID) → Grade

5. Verify 2NF

In STUDENT, SID is the complete primary key, so SName fully depends on SID.

In COURSE, CourseID is the complete primary key, so CourseName and Faculty fully depend on CourseID.

In ENROLLMENT, Grade depends on the complete composite key (SID, CourseID).

Therefore, there are **no partial dependencies** in any of the decomposed relations.

Final 2NF Decomposition:

STUDENT(SID, SName)

COURSE(CourseID, CourseName, Faculty)

ENROLLMENT(SID, CourseID, Grade)

Conclusion:

The original ENROLLMENT relation has the candidate key **(SID, CourseID)** and contains partial dependencies **SID → SName** and **CourseID → CourseName, Faculty**. By decomposing it into **STUDENT, COURSE, and ENROLLMENT**, all partial dependencies are eliminated. Hence, the resulting relations are in **2NF**.

4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

BCNF Normalization of Airline Booking Relation

Given Relation:

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

The relation stores flight, passenger, seat, and travel information in a single relation.

1. Identify the Functional Dependencies

Assume the following business rules:

PassID → PassName

A passenger ID uniquely identifies the passenger name.

FlightNo → DeptCity, ArrCity

A flight number uniquely identifies its departure and arrival cities.

(FlightNo, Date, Seat) → PassID

For a particular flight on a particular date, a seat is assigned to only one passenger.

(FlightNo, Date, PassID) → Seat

For a particular flight on a particular date, a passenger is assigned only one seat.

Therefore, the important functional dependencies are:

PassID → PassName

FlightNo → DeptCity, ArrCity

(FlightNo, Date, Seat) → PassID

(FlightNo, Date, PassID) → Seat

2. Identify the Candidate Keys

Consider:

(FlightNo, Date, Seat)

Using:

(FlightNo, Date, Seat) → PassID

Then:

PassID → PassName

And:

FlightNo → DeptCity, ArrCity

Therefore:

(FlightNo, Date, Seat)+ = {FlightNo, Date, Seat, PassID, PassName, DeptCity, ArrCity}

Thus, **(FlightNo, Date, Seat)** is a candidate key.

Now consider:

(FlightNo, Date, PassID)

Using:

(FlightNo, Date, PassID) → Seat

Then:

PassID → PassName

And:

FlightNo → DeptCity, ArrCity

Therefore:

(FlightNo, Date, PassID)+ = {FlightNo, Date, PassID, Seat, PassName, DeptCity, ArrCity}

Thus, **(FlightNo, Date, PassID)** is also a candidate key.

Candidate Keys:

(FlightNo, Date, Seat)

(FlightNo, Date, PassID)

3. Check for BCNF

A relation is in BCNF if, for every non-trivial functional dependency X → Y, **X must be a superkey**.

Consider:

PassID → PassName

PassID is not a superkey because it cannot determine FlightNo, Date, or Seat.

Therefore, this FD violates BCNF.

Consider:

FlightNo → DeptCity, ArrCity

FlightNo is not a superkey because it cannot determine PassID, Seat, or Date.

Therefore, this FD also violates BCNF.

The dependencies:

(FlightNo, Date, Seat) → PassID

(FlightNo, Date, PassID) → Seat

satisfy BCNF because their determinants are candidate keys.

4. Decompose the Relation

To remove the BCNF violation caused by:

PassID → PassName

create:

PASSENGER(PassID, PassName)

Primary Key: **PassID**

For:

FlightNo → DeptCity, ArrCity

create:

FLIGHT(FlightNo, DeptCity, ArrCity)

Primary Key: **FlightNo**

The remaining booking information is:

BOOKING(FlightNo, Date, PassID, Seat)

Candidate Keys:

(FlightNo, Date, PassID)

(FlightNo, Date, Seat)

5. Verify BCNF

PASSENGER:

PassID → PassName

PassID is the candidate key, so PASSENGER is in BCNF.

FLIGHT:

FlightNo → DeptCity, ArrCity

FlightNo is the candidate key, so FLIGHT is in BCNF.

BOOKING:

(FlightNo, Date, PassID) → Seat

(FlightNo, Date, Seat) → PassID

Both determinants are candidate keys, so BOOKING is in BCNF.

Final BCNF Decomposition:

PASSENGER(PassID, PassName)

Primary Key: PassID

FLIGHT(FlightNo, DeptCity, ArrCity)

Primary Key: FlightNo

BOOKING(FlightNo, Date, PassID, Seat)

Primary Keys: (FlightNo, Date, PassID) and (FlightNo, Date, Seat)

Conclusion:

The original BOOKING relation is not in BCNF because **PassID → PassName** and **FlightNo → DeptCity, ArrCity** have determinants that are not superkeys. After decomposition into **PASSENGER, FLIGHT, and BOOKING**, every non-trivial functional dependency has a superkey as its determinant. Therefore, the resulting relations are in **BCNF**.

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

3NF Normalization of HR System Schema

Given Relation:

Consider the HR relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, LocationID, LocationName, Salary)

This relation stores employee, department, manager, and location information in one table.

1. Identify Functional Dependencies

The main functional dependencies are:

EmpID → EmpName, DeptID, ManagerID, Salary

An employee ID uniquely identifies the employee's name, department, manager, and salary.

DeptID → DeptName, LocationID

A department ID uniquely identifies the department name and its location.

LocationID → LocationName

A location ID uniquely identifies the location name.

ManagerID → ManagerName

A manager ID uniquely identifies the manager's name.

Therefore, the complete set of important functional dependencies is:

EmpID → EmpName, DeptID, ManagerID, Salary

DeptID → DeptName, LocationID

LocationID → LocationName

ManagerID → ManagerName

2. Identify the Candidate Key

Consider the closure of EmpID:

EmpID+ = {EmpID}

Using:

EmpID → EmpName, DeptID, ManagerID, Salary

we obtain:

EmpID+ = {EmpID, EmpName, DeptID, ManagerID, Salary}

Using:

DeptID → DeptName, LocationID

we obtain:

EmpID+ = {EmpID, EmpName, DeptID, DeptName, ManagerID, LocationID, Salary}

Using:

LocationID → LocationName

we obtain LocationName.

Using:

ManagerID → ManagerName

we obtain ManagerName.

Therefore:

EmpID+ = {EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, LocationID, LocationName, Salary}

Since EmpID determines all attributes:

Candidate Key = EmpID

3. Identify Transitive Dependencies

A transitive dependency occurs when a non-key attribute determines another non-key attribute.

The first transitive dependency is:

EmpID → DeptID → DeptName, LocationID

Therefore:

EmpID → DeptName, LocationID

The second transitive dependency is:

EmpID → LocationID → LocationName

Therefore:

EmpID → LocationName

The third transitive dependency is:

EmpID → ManagerID → ManagerName

Therefore:

EmpID → ManagerName

Thus, the important transitive dependencies are:

EmpID → DeptID → DeptName, LocationID

EmpID → LocationID → LocationName

EmpID → ManagerID → ManagerName

These dependencies violate 3NF because non-key attributes determine other non-key attributes.

4. Decompose the Relation

To eliminate the transitive dependencies, create separate relations.

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, Salary)

Primary Key: **EmpID**

Functional dependency:

EmpID → EmpName, DeptID, ManagerID, Salary

DEPARTMENT(DeptID, DeptName, LocationID)

Primary Key: **DeptID**

Functional dependency:

DeptID → DeptName, LocationID

LOCATION(LocationID, LocationName)

Primary Key: **LocationID**

Functional dependency:

LocationID → LocationName

MANAGER(ManagerID, ManagerName)

Primary Key: **ManagerID**

Functional dependency:

ManagerID → ManagerName

5. Foreign Keys

EMPLOYEE.DeptID references DEPARTMENT.DeptID.

EMPLOYEE.ManagerID references MANAGER.ManagerID.

DEPARTMENT.LocationID references LOCATION.LocationID.

6. Verify 3NF

In EMPLOYEE:

EmpID → EmpName, DeptID, ManagerID, Salary

EmpID is the primary key, so the relation satisfies 3NF.

In DEPARTMENT:

DeptID → DeptName, LocationID

DeptID is the primary key, so the relation satisfies 3NF.

In LOCATION:

LocationID → LocationName

LocationID is the primary key, so the relation satisfies 3NF.

In MANAGER:

ManagerID → ManagerName

ManagerID is the primary key, so the relation satisfies 3NF.

Therefore, all resulting relations are in **3NF**.

Final 3NF Schema:

EMPLOYEE(EmpID, EmpName, DeptID, ManagerID, Salary)

DEPARTMENT(DeptID, DeptName, LocationID)

LOCATION(LocationID, LocationName)

MANAGER(ManagerID, ManagerName)

Conclusion:

The original HR relation contains functional dependencies such as **EmpID → EmpName, DeptID, ManagerID, Salary** and transitive dependencies such as **EmpID → DeptID → DeptName** and **EmpID → ManagerID → ManagerName**. By decomposing the schema into EMPLOYEE, DEPARTMENT, LOCATION, and MANAGER, the transitive dependencies are eliminated. The resulting relations have no partial or transitive dependencies and therefore satisfy **Third Normal Form (3NF)**.

6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

4NF Decomposition of LIBRARY Relation

Given Relation:

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

The relation stores member, book, author, genre, and borrowing information together.

1. Identify the Functional Dependencies

Assume the following dependencies:

MemberID → MemberName

A MemberID uniquely identifies the member name.

BookID → Title, Author, Genre

A BookID uniquely identifies the book title, author, and genre.

(MemberID, BookID) → BorrowDate

The borrowing date is determined by the combination of a member and a book.

Therefore, the candidate key is:

(MemberID, BookID)

2. Identify the Multi-Valued Dependency

Suppose a member can borrow multiple books, and each book can have multiple independent attributes such as authors or genres.

For the purpose of the 4NF case, assume that a member can have multiple independent **books** and **genres**.

The important MVD is:

MemberID →→ BookID

This means that the set of books associated with a member is independent of other multivalued information associated with that member.

However, the given relation does not explicitly contain another independent multivalued attribute besides BookID. Therefore, **strictly speaking, an MVD violation cannot be established from the given schema alone** without an additional business rule.

3. Standard 4NF Case Assumption

If the intended case is that each member can independently have multiple borrowed books and multiple genres of interest, then:

MemberID →→ BookID

MemberID →→ Genre

These independent multivalued dependencies cause unnecessary combinations.

For example, if Member M01 has:

Books: B01, B02

Genres: Fiction, Science

the relation may generate:

M01, B01, Fiction

M01, B01, Science

M01, B02, Fiction

M01, B02, Science

This creates redundancy.

4. 4NF Decomposition

To eliminate the independent multivalued dependencies, decompose the relation into:

MEMBER(MemberID, MemberName)

Primary Key: **MemberID**

BOOK(BookID, Title, Author, Genre)

Primary Key: **BookID**

BORROW(MemberID, BookID, BorrowDate)

Primary Key: **(MemberID, BookID)**

5. Verify 4NF

MEMBER contains only member information.

BOOK contains only book information.

BORROW contains the relationship between a member and a book along with the borrowing date.

Therefore, the independent multivalued information is separated, eliminating the 4NF violation.

Final 4NF Decomposition:

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, Author, Genre)

BORROW(MemberID, BookID, BorrowDate)

Conclusion:

The given LIBRARY relation contains functional dependencies such as **MemberID → MemberName** and **BookID → Title, Author, Genre**. For a genuine 4NF decomposition, an independent multi-valued relationship must exist. Under the standard assumption that members can independently be associated with multiple books and other independent values, the relation is decomposed into **MEMBER, BOOK, and BORROW**. This reduces redundancy and organizes the database into relations satisfying **4NF**.

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Telecom Billing Schema – Functional Dependency and Key Analysis

Consider the following telecom billing relation:

BILLING(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallID, CallDate, Duration, CallCharge)

This schema stores customer, mobile plan, and individual call billing information in one relation.

1. Functional Dependencies

The functional dependencies can be identified as follows:

CustomerID → CustomerName, PhoneNo, PlanID

A CustomerID uniquely identifies the customer, phone number, and subscribed plan.

PhoneNo → CustomerID

Assuming each phone number belongs to exactly one customer, PhoneNo uniquely identifies the customer.

PlanID → PlanName, MonthlyCharge

A PlanID uniquely identifies the plan name and monthly charge.

CallID → CustomerID, CallDate, Duration, CallCharge

A CallID uniquely identifies a particular call and its details.

Therefore, the main functional dependencies are:

CustomerID → CustomerName, PhoneNo, PlanID

PhoneNo → CustomerID

PlanID → PlanName, MonthlyCharge

CallID → CustomerID, CallDate, Duration, CallCharge

2. Determine the Candidate Keys

A candidate key must uniquely identify every tuple in the relation.

Since the relation contains individual call information, **CallID** can uniquely identify a complete record.

Therefore:

CallID+ = {CallID, CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallDate, Duration, CallCharge}

Hence:

CallID is a candidate key.

Under the given assumptions, no other single attribute determines all attributes.

Therefore:

Candidate Key = CallID

3. Primary Key

Since CallID is the only candidate key, it can be selected as the primary key.

Primary Key = CallID

4. Candidate Key Verification

CallID → CustomerID, CallDate, Duration, CallCharge

CustomerID → CustomerName, PhoneNo, PlanID

PlanID → PlanName, MonthlyCharge

Therefore, through transitivity:

CallID → CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge

Thus, CallID determines every attribute in the relation.

5. Transitive Dependencies

There are several transitive dependencies:

CallID → CustomerID → CustomerName

CallID → CustomerID → PhoneNo

CallID → CustomerID → PlanID

CallID → PlanID → PlanName

CallID → PlanID → MonthlyCharge

These indicate that customer and plan information is unnecessarily repeated for every call.

6. Primary Key and Candidate Key Summary

Candidate Key: CallID

Primary Key: CallID

Non-key attributes: CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallDate, Duration, CallCharge

7. Important Note

The exact candidate keys depend on the business rules of the telecom system. For example, if CallID is not globally unique and a call is identified by **(PhoneNo, CallDate, Duration)**, that combination could potentially become a candidate key. Therefore, the above analysis assumes that **CallID uniquely identifies every call**.

Conclusion:

For the assumed telecom billing schema:

BILLING(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallID, CallDate, Duration, CallCharge)

the main functional dependencies are:

CustomerID → CustomerName, PhoneNo, PlanID

PhoneNo → CustomerID

PlanID → PlanName, MonthlyCharge

CallID → CustomerID, CallDate, Duration, CallCharge

The **candidate key is CallID**, and it is selected as the **primary key**. The schema also contains several transitive dependencies, indicating that normalization would be useful to reduce redundancy.

8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Complete Normalization of RETAIL INVENTORY Relation

Given Relation:

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

The relation stores product, supplier, category, price, and warehouse information together. This can lead to redundancy and insertion, deletion, and update anomalies.

1. Identify Functional Dependencies

Assume the following business rules:

PID → PName, SupplierID, Category, Price, Warehouse

A Product ID uniquely identifies the product and its associated details.

SupplierID → SupplierName

A Supplier ID uniquely identifies the supplier name.

Therefore, the main functional dependencies are:

PID → PName, SupplierID, Category, Price, Warehouse

SupplierID → SupplierName

2. Identify the Candidate Key

Since PID determines all attributes:

PID+ = {PID, PName, SupplierID, SupplierName, Category, Price, Warehouse}

Therefore:

Candidate Key = PID

Primary Key = PID

3. Check 1NF

The relation is assumed to contain atomic values.

For example, each product has a single PID, PName, SupplierID, Category, Price, and Warehouse value.

Therefore, the relation is in **1NF**.

4. Check 2NF

The candidate key is a single attribute:

PID

Since the candidate key is not composite, partial dependency cannot exist.

Therefore, the relation is already in **2NF**.

5. Check 3NF

Consider:

PID → SupplierID

and:

SupplierID → SupplierName

Therefore:

PID → SupplierID → SupplierName

This is a **transitive dependency** because SupplierID is a non-key attribute determining another non-key attribute, SupplierName.

Therefore, the original relation is **not in 3NF**.

6. Decompose into 3NF

Separate supplier information from product information.

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: **PID**

Foreign Key: **SupplierID**

Functional dependency:

PID → PName, SupplierID, Category, Price, Warehouse

Create a separate supplier relation:

SUPPLIER(SupplierID, SupplierName)

Primary Key: **SupplierID**

Functional dependency:

SupplierID → SupplierName

7. Check BCNF

In PRODUCT:

PID → PName, SupplierID, Category, Price, Warehouse

PID is the candidate key, so PRODUCT satisfies BCNF.

In SUPPLIER:

SupplierID → SupplierName

SupplierID is the candidate key, so SUPPLIER satisfies BCNF.

Therefore, the final decomposition is not only in 3NF but also satisfies **BCNF**, under the stated assumptions.

8. Final Normalized Schema

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: **PID**

Foreign Key: **SupplierID**

SUPPLIER(SupplierID, SupplierName)

Primary Key: **SupplierID**

Conclusion:

The original PRODUCT relation is in **1NF and 2NF**, but not in **3NF** because of the transitive dependency:

PID → SupplierID → SupplierName

After decomposing it into **PRODUCT** and **SUPPLIER**, the transitive dependency is removed. The resulting relations satisfy **3NF and BCNF**, completing the normalization process under the given functional dependencies.

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Fully Normalized School ERP Design up to BCNF

Given Poorly Normalized Schema:

Consider the following School ERP relation:

SCHOOL_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ParentID, ParentName, PhoneNo)

This single relation stores student, class, teacher, subject, marks, and parent information together. Such a design produces significant redundancy and can cause insertion, deletion, and update anomalies.

1. Identify the Functional Dependencies

Based on the meaning of the attributes, assume the following functional dependencies:

StudentID → StudentName, ClassID, ParentID

StudentID uniquely identifies a student, their class, and their parent.

ClassID → ClassName, TeacherID

ClassID uniquely identifies the class and its class teacher.

TeacherID → TeacherName

TeacherID uniquely identifies the teacher.

SubjectID → SubjectName

SubjectID uniquely identifies the subject.

ParentID → ParentName, PhoneNo

ParentID uniquely identifies the parent and contact number.

(StudentID, SubjectID) → Marks

The marks obtained by a student are determined by the combination of StudentID and SubjectID.

Therefore, the important FDs are:

StudentID → StudentName, ClassID, ParentID

ClassID → ClassName, TeacherID

TeacherID → TeacherName

SubjectID → SubjectName

ParentID → ParentName, PhoneNo

(StudentID, SubjectID) → Marks

2. Identify the Candidate Key

Consider:

(StudentID, SubjectID)+

From:

StudentID → StudentName, ClassID, ParentID

we obtain StudentName, ClassID, and ParentID.

From:

ClassID → ClassName, TeacherID

we obtain ClassName and TeacherID.

From:

TeacherID → TeacherName

we obtain TeacherName.

From:

ParentID → ParentName, PhoneNo

we obtain ParentName and PhoneNo.

From:

SubjectID → SubjectName

we obtain SubjectName.

From:

(StudentID, SubjectID) → Marks

we obtain Marks.

Therefore:

(StudentID, SubjectID)+ = {StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ParentID, ParentName, PhoneNo}

Thus, **(StudentID, SubjectID)** is a candidate key.

3. Identify the Anomalies

Insertion Anomaly:

A new subject cannot easily be added unless at least one student is enrolled in that subject.

A new teacher or parent may also require an associated student record.

Deletion Anomaly:

If the only student taking a particular subject is deleted, information about that subject may also be lost.

Similarly, deleting the only student belonging to a class could remove class or teacher information.

Update Anomaly:

If a teacher's name changes, multiple student records may need to be updated.

If a parent changes their phone number, the value may have to be updated in several records.

This can lead to inconsistent data.

4. Check 1NF

The relation contains atomic values and no repeating groups.

Therefore, the relation is assumed to be in **1NF**.

5. Convert to 2NF

The candidate key is:

(StudentID, SubjectID)

However:

StudentID → StudentName, ClassID, ParentID

These attributes depend only on StudentID, which is part of the composite key.

Also:

SubjectID → SubjectName

SubjectName depends only on SubjectID, which is another part of the composite key.

Therefore, partial dependencies exist and the relation is not in 2NF.

Decompose into:

STUDENT(StudentID, StudentName, ClassID, ParentID)

SUBJECT(SubjectID, SubjectName)

STUDENT_SUBJECT(StudentID, SubjectID, Marks)

6. Identify Transitive Dependencies

In STUDENT:

StudentID → ClassID

and:

ClassID → ClassName, TeacherID

Therefore:

StudentID → ClassID → ClassName, TeacherID

Also:

TeacherID → TeacherName

Therefore:

StudentID → ClassID → TeacherID → TeacherName

Similarly:

StudentID → ParentID

and:

ParentID → ParentName, PhoneNo

Therefore:

StudentID → ParentID → ParentName, PhoneNo

These are transitive dependencies and must be removed to achieve 3NF.

7. Convert to 3NF

Create separate relations:

STUDENT(StudentID, StudentName, ClassID, ParentID)

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

PARENT(ParentID, ParentName, PhoneNo)

SUBJECT(SubjectID, SubjectName)

STUDENT_SUBJECT(StudentID, SubjectID, Marks)

Now, each non-key attribute depends directly on the key of its relation.

8. Check BCNF

A relation is in BCNF if, for every non-trivial FD X → Y, X is a superkey.

STUDENT

StudentID → StudentName, ClassID, ParentID

StudentID is the candidate key.

Therefore, STUDENT satisfies BCNF.

CLASS

ClassID → ClassName, TeacherID

ClassID is the candidate key.

Therefore, CLASS satisfies BCNF.

TEACHER

TeacherID → TeacherName

TeacherID is the candidate key.

Therefore, TEACHER satisfies BCNF.

PARENT

ParentID → ParentName, PhoneNo

ParentID is the candidate key.

Therefore, PARENT satisfies BCNF.

SUBJECT

SubjectID → SubjectName

SubjectID is the candidate key.

Therefore, SUBJECT satisfies BCNF.

STUDENT_SUBJECT

(StudentID, SubjectID) → Marks

(StudentID, SubjectID) is the candidate key.

Therefore, STUDENT_SUBJECT satisfies BCNF.

9. Final BCNF Schema

STUDENT(StudentID, StudentName, ClassID, ParentID)

Primary Key: StudentID

Foreign Keys: ClassID, ParentID

CLASS(ClassID, ClassName, TeacherID)

Primary Key: ClassID

Foreign Key: TeacherID

TEACHER(TeacherID, TeacherName)

Primary Key: TeacherID

PARENT(ParentID, ParentName, PhoneNo)

Primary Key: ParentID

SUBJECT(SubjectID, SubjectName)

Primary Key: SubjectID

STUDENT_SUBJECT(StudentID, SubjectID, Marks)

Primary Key: (StudentID, SubjectID)

Foreign Keys: StudentID, SubjectID

10. Normalization Summary

UNF/1NF:

SCHOOL_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ParentID, ParentName, PhoneNo)

↓

2NF:

STUDENT(StudentID, StudentName, ClassID, ParentID)

SUBJECT(SubjectID, SubjectName)

STUDENT_SUBJECT(StudentID, SubjectID, Marks)

↓

3NF:

STUDENT(StudentID, StudentName, ClassID, ParentID)

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

PARENT(ParentID, ParentName, PhoneNo)

SUBJECT(SubjectID, SubjectName)

STUDENT_SUBJECT(StudentID, SubjectID, Marks)

↓

BCNF:

All the above relations satisfy BCNF because every determinant in each relation is a candidate key.

Conclusion:

The original School ERP schema combines several independent entities into one large relation, causing redundancy and insertion, deletion, and update anomalies. The normalization process first removes partial dependencies in **2NF**, then removes transitive dependencies in **3NF**. The final decomposition into **STUDENT, CLASS, TEACHER, PARENT, SUBJECT, and STUDENT_SUBJECT** ensures that every non-trivial functional

dependency has a candidate key as its determinant. Hence, the final design satisfies **BCNF**, providing better consistency, reduced redundancy, and easier database maintenance.

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

5NF Decomposition of Movie Streaming Platform Schema

Given Relation:

Consider the following poorly normalized relation:

STREAMING(MovieID, MovieTitle, ActorID, ActorName, PlatformID, PlatformName)

Assume that a movie can have multiple actors and can be available on multiple streaming platforms. The relationship between Movie, Actor, and Platform is independent.

1. Identify the Functional Dependencies

The functional dependencies are:

MovieID → MovieTitle

MovieID uniquely identifies the movie title.

ActorID → ActorName

ActorID uniquely identifies the actor name.

PlatformID → PlatformName

PlatformID uniquely identifies the streaming platform name.

The actual association is represented by:

(MovieID, ActorID, PlatformID)

This combination identifies a movie-actor-platform association.

2. Identify the Join Dependency

The three-way relationship can be represented using three pairwise relationships:

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Therefore, the original relation has the join dependency:

(MovieID, ActorID), (MovieID, PlatformID), (ActorID, PlatformID)

This means the original relation can be reconstructed by joining these three projections.

3. Demonstrate Redundancy

Suppose:

Movie M01 has actors A01 and A02.

Movie M01 is available on platforms P01 and P02.

The combined relation may contain:

M01, A01, P01

M01, A01, P02

M01, A02, P01

M01, A02, P02

The movie and actor/platform relationships are repeated unnecessarily.

For example, the fact that M01 has actor A01 is repeated for every platform on which the movie is available.

This causes redundancy.

4. Decompose into 5NF

Decompose the relation into:

MOVIE(MovieID, MovieTitle)

ACTOR(ActorID, ActorName)

PLATFORM(PlatformID, PlatformName)

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

The three relationship relations remove the independent many-to-many combinations from the original relation.

5. Lossless-Join Verification

The original relation can be reconstructed as:

MOVIE_ACTOR ⋈ MOVIE_PLATFORM ⋈ ACTOR_PLATFORM

along with the descriptive relations:

MOVIE ⋈ ACTOR ⋈ PLATFORM

If the business rule states that the three pairwise relationships completely determine the movie-actor-platform relationship, then joining the decomposed relations produces exactly the original tuples.

For example:

MOVIE_ACTOR:

M01, A01
M01, A02

MOVIE_PLATFORM:

M01, P01
M01, P02

ACTOR_PLATFORM:

A01, P01
A01, P02
A02, P01
A02, P02

The join produces:

M01, A01, P01

M01, A01, P02

M01, A02, P01

M01, A02, P02

Thus, the original relation can be reconstructed without losing information.

6. 5NF Verification

A relation is in **5NF (Project-Join Normal Form)** when every non-trivial join dependency is implied by the candidate keys.

After decomposition, the independent relationships are stored separately. No further non-trivial decomposition is required under the stated business rules.

Therefore, the resulting design satisfies **5NF**.

Final 5NF Schema:

MOVIE(MovieID, MovieTitle)

ACTOR(ActorID, ActorName)

PLATFORM(PlatformID, PlatformName)

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Conclusion:

The original movie streaming relation contains a three-way relationship that can create redundancy when movie-actor and movie-platform relationships are stored together. By decomposing it into separate movie, actor, platform, and relationship relations, the join dependency is isolated. Under the stated independence assumption, the decomposition is **lossless-join** and satisfies **5NF (Project-Join Normal Form)**.